

Tutorial - Semana 6, Encontro 2

Introdução

No Encontro 1, o projeto ganhou o Resource customizado `ItemData`, com campos como `nome`, `icone`, `valor` e `descricao`, e pelo menos duas instâncias `.tres` de itens distintos. Este encontro completa o par que resolve o desacoplamento entre dado e lógica: o **Enum**. Até agora, uma eventual categoria de item (moeda, chave, recurso) só poderia ser representada por uma `String` livre — sujeita a erro de digitação e sem garantia de consistência entre instâncias. O Enum resolve isso restringindo o campo a um conjunto fechado e nomeado de valores.

Este tutorial dá continuidade direta ao Encontro 1 — a classe `ItemData` e as instâncias `.tres` já devem existir antes de começar.

Objetivos da semana

- Explicar Enums como organizadores de dados tipados dentro de um Resource.
- Diferenciar um campo tipado por Enum de um campo livre (string ou número mágico).
- Modelar um conjunto próprio de itens coletáveis usando `ItemData` + Enum.

Resultado esperado ao final da semana

Ao final da Semana 6 (Encontros 1 e 2), cada grupo terá uma classe `ItemData` com pelo menos um Enum de categoria, aplicada a um conjunto próprio de três ou mais itens coletáveis. Este tutorial cobre apenas o **Encontro 2**: a declaração do Enum, sua aplicação ao `ItemData`, e o desafio de cada grupo modelar seu próprio conjunto de itens — que compõe o Checkpoint de progresso do Módulo 2.

Pré-requisitos

- Classe `ItemData` e pelo menos duas instâncias `.tres` do Encontro 1, já testadas (ver Tutorial - Semana 6, Encontro 1).
-

Antes de começar

O que o estudante deverá possuir antes desta semana

- O projeto do Encontro 1, com a classe `ItemData` definida em `scripts/resources/item_data.gd` e pelo menos duas instâncias `.tres` em `resources/items/`.

Arquivos necessários

- Nenhum arquivo externo adicional.

Assets utilizados

- Os mesmos ícones opcionais já utilizados (ou não) no Encontro 1.

Projeto esperado

- Projeto aberto no Godot 4.7, com `ItemData` e as instâncias `.tres` do Encontro 1 já testadas.

Imagem sugerida

Objetivo: mostrar o campo `categoria` do `ItemData` no Inspector, exibido como menu suspenso gerado a partir do Enum. Enquadramento: captura de tela do Inspector com uma instância `.tres` de item selecionada. Elementos importantes: o campo `categoria`, o menu suspenso aberto mostrando as opções do Enum (por exemplo, MOEDA, RECURSO, CHAVE). Destaque: o menu suspenso, contrastando com um campo de texto livre. Legenda sugerida: "Campo categoria do `ItemData` exibido como menu suspenso gerado pelo Enum."

Parte 1 — Enums como conjunto fechado de valores

Objetivo

Entender por que um campo de categoria de item não deve ser uma `String` livre.

Conceito

O `ItemData` do Encontro 1 resolve "onde vive o dado"; o Enum resolve um problema complementar: "como impedir que um campo de categoria aceite qualquer valor digitado livremente, sujeito a erro de digitação ou inconsistência entre itens". Se `categoria` fosse uma `String`, nada impediria que um item recebesse `"moeda"`, outro `"Moeda"` e outro `"moedas"` — três valores diferentes para a mesma ideia, invisíveis até gerarem um bug de comparação em tempo de execução.

Enums existem em praticamente toda linguagem de programação porque o mesmo problema — um conjunto fechado e conhecido de opções — aparece em qualquer domínio, não apenas em jogos. No Godot, um Enum é declarado dentro do script do Resource e passa a aparecer como um menu suspenso no Inspector, eliminando a possibilidade de um valor fora do conjunto esperado.

Passo a passo

Esta parte não possui etapas de implementação no editor — é a base conceitual antes da Parte 2.

1. Revisar com a turma o campo `nome` do `ItemData` (uma `String` livre, adequada porque cada item tem um nome único) em contraste com uma eventual `categoria` (um conjunto pequeno e repetido entre itens).
2. Perguntar: "se dez itens diferentes puderem pertencer a apenas três categorias, faz sentido cada um digitar a categoria como texto livre?"
3. Concluir que a resposta correta é não — um conjunto pequeno e fechado de opções deve ser um Enum, nunca uma `String`.

Resultado esperado

A turma entende por que um campo de categoria pertence a um Enum, e não a uma `String` livre ou a um número mágico sem nome.

Verificando

1. Confirmar que os estudantes conseguem citar, em suas próprias palavras, um exemplo de campo do `ItemData` que deveria ser Enum e outro que deveria continuar `String`.

Problemas comuns

- Achar que todo campo do `ItemData` deveria virar Enum: reforçar que Enum serve apenas para conjuntos fechados e pequenos de opções repetidas entre instâncias; `nome` e `descricao` continuam `String` porque são únicos por item.

Boas práticas

- Manter esta discussão breve, reservando a maior parte do encontro para a implementação nas Partes 2 e 3.

Comparação com Unity

A Unity resolve o mesmo padrão com `enum` de C#, aplicado a um campo público ou `[SerializeField]` de um `ScriptableObject` — mecanismo praticamente idêntico ao do Godot: declaração de um conjunto fechado de valores nomeados, exposto no Inspector como menu suspenso. A diferença relevante não está no conceito, mas em GDScript tratar Enums como valores inteiros nomeados dentro do próprio script, sem exigir um arquivo de definição separado — o C# permite (mas não exige) declarar o `enum` fora da classe que o usa. O conceito universal — restringir um campo a um conjunto fechado e nomeado de opções — é idêntico nas duas engines.

Parte 2 — Declarando o Enum de categoria no `ItemData`

Objetivo

Adicionar um Enum de categoria ao `ItemData` e aplicá-lo às instâncias já criadas no Encontro 1.

Conceito

Um Enum em GDScript é declarado com a palavra-chave `enum`, seguindo PascalCase para o nome do tipo, conforme o PROJECT_ARCHITECTURE.md (seção 9). Ao declarar um campo `@export` do tipo do Enum, o Godot gera automaticamente um menu suspenso no Inspector com as opções nomeadas — sem exigir nenhuma configuração adicional além da declaração no script.

Passo a passo

1. Abra o script `item_data.gd`, criado no Encontro 1.
2. Logo abaixo de `class_name ItemData`, declare o Enum de categoria:

```
enum Categoria { MOEDA, RECURSO, CHAVE }
```

3. Adicione um novo campo exportado do tipo do Enum:

```
@export var categoria: Categoria = Categoria.MOEDA
```

4. Salve o script (**Ctrl+S**).
5. No FileSystem Dock, selecione a instância `item_moeda.tres` criada no Encontro 1.
6. No Inspector, localize o novo campo `categoria` e confirme que ele aparece como menu suspenso com as opções `MOEDA`, `RECURSO` e `CHAVE`.
7. Defina `categoria` como `MOEDA` para `item_moeda.tres` e `CHAVE` para `item_chave.tres`.
8. Salve ambas as instâncias (**Ctrl+S** com cada uma selecionada, ou **Salvar Tudo**).

Resultado esperado

Cada instância `.tres` de item passa a ter um campo `categoria`, preenchido a partir do Enum `Categoria`, visível como menu suspenso no Inspector — sem nenhuma possibilidade de digitar um valor fora do conjunto declarado.

Verificando

1. Selecione `item_moeda.tres` e confirme que `categoria` está definido como `MOEDA`.
2. Selecione `item_chave.tres` e confirme que `categoria` está definido como `CHAVE`.
3. Tente, no Inspector, alterar `categoria` e confirme que apenas as três opções do Enum estão disponíveis — nenhum campo de texto livre.

Problemas comuns

- Usar uma `String` livre em vez do Enum para representar a categoria, reintroduzindo o risco de inconsistência que o Enum existe para evitar: reforçar que toda categoria fechada e conhecida de antemão deve ser um Enum, não uma `String`.
- Criar um Enum excessivamente granular (uma categoria por item individual, por exemplo `MOEDA_OURO`, `MOEDA_PRATA` como categorias separadas em vez de uma categoria `MOEDA` com um campo `valor` distinto): orientar a pensar em categorias, não em identificadores únicos.
- Esquecer de salvar a instância `.tres` após alterar o campo `categoria` no Inspector, perdendo a alteração ao trocar de seleção: orientar salvamento explícito de cada instância alterada.

Boas práticas

- Nomear o Enum em PascalCase (`Categoria`) e seus valores em maiúsculas (`MOEDA`, `RECURSO`, `CHAVE`), conforme a seção 9 do `PROJECT_ARCHITECTURE.md`.
- Definir um valor padrão coerente no campo `@export` (`Categoria.MOEDA`), evitando instâncias criadas sem categoria explícita.
- Manter o Enum pequeno (três a cinco categorias) — um Enum com dezenas de valores geralmente indica que o conceito deveria ser outro campo, não uma categoria.

Comparação com Unity

Na Unity, o mesmo resultado exigiria declarar um `enum Categoria { Moeda, Recurso, Chave }` em C# e um campo `[SerializeField] private Categoria categoria;` dentro do `ScriptableObject`. O comportamento é equivalente — um menu suspenso gerado automaticamente no Inspector —, mas a Unity, por convenção C#, usa PascalCase também para os valores do enum (`Moeda`, não `MOEDA`), enquanto o GDScript segue a convenção de constantes em maiúsculas. O conceito de restringir o campo a um conjunto fechado é idêntico nas duas engines.

Parte 3 — Desafio: conjunto próprio de itens coletáveis

Objetivo

Cada grupo modela seu próprio conjunto de itens coletáveis usando `ItemData` + Enum, com liberdade de categorias e atributos.

Conceito

A combinação `ItemData` + Enum construída nas Partes 1 e 2 não é exclusiva das categorias `MOEDA`, `RECURSO` e `CHAVE` demonstradas: cada grupo pode ajustar o Enum `Categoria` ao tema do próprio Vertical Slice, mantendo a mesma estrutura de campos exportados e o mesmo princípio de dado desacoplado de lógica. Este é o teste real do conceito ensinado nesta semana — um conjunto de itens completamente diferente do demonstrado deve se encaixar na mesma classe `ItemData`, sem exigir nenhuma alteração na estrutura do Resource.

Passo a passo

1. Em grupo, revise o Enum `Categoria` demonstrado e decida se as três categorias (`MOEDA`, `RECURSO`, `CHAVE`) atendem ao tema do próprio Vertical Slice ou se precisam de ajuste (por exemplo, `CONSUMIVEL`, `EQUIPAMENTO`, `QUEST_ITEM`).
2. Se necessário, edite o Enum em `item_data.gd`, adicionando ou renomeando valores — mantendo o conjunto pequeno (três a cinco categorias).
3. Crie pelo menos três instâncias `.tres` de itens em `resources/items/`, preenchendo `nome`, `icone` (se houver), `valor`, `descricao` e `categoria` para cada uma.
4. Nomeie cada arquivo `.tres` descrevendo o item concreto (por exemplo, `item_pocao_cura.tres`), conforme a seção 9 do PROJECT_ARCHITECTURE.md.
5. Confirme, para cada instância, que o campo `categoria` reflete corretamente o Enum ajustado pelo grupo.
6. Prepare uma apresentação rápida (1 a 2 minutos) do conjunto de itens do grupo para o Checkpoint de progresso ao final do encontro.

Resultado esperado

Cada grupo possui, ao final da semana, uma classe `ItemData` com pelo menos um Enum de categoria e um conjunto de três ou mais instâncias de itens distintos, coerentes com o tema do próprio Vertical Slice.

Verificando

1. Confirme que todas as instâncias `.tres` do grupo estão salvas em `resources/items/`.
2. Selecione cada instância e confirme, no Inspector, que `categoria` está preenchido a partir do menu suspenso do Enum — nenhum campo de texto livre substituindo essa função.
3. Confirme que os nomes dos arquivos `.tres` descrevem o item concreto, seguindo a convenção `snake_case` do projeto.

Problemas comuns

- Duplicar a classe `ItemData` para cada grupo em vez de reutilizar a mesma estrutura ajustando apenas o Enum e as instâncias: reforçar que a classe é única no projeto — apenas o Enum interno e as instâncias variam por grupo.
- Criar itens sem preencher o campo `categoria`, deixando-o no valor padrão por engano: orientar verificação de cada instância antes da apresentação do Checkpoint.

Boas práticas

- Escolher categorias que reflitam decisões de gameplay já tomadas pelo grupo (por exemplo, se o Vertical Slice do grupo já tem uma mecânica de crafting, incluir uma categoria `MATERIAL`).
- Revisar o conjunto de itens do grupo antes da apresentação, garantindo que nenhuma instância ficou incompleta.

Comparação com Unity

O mesmo desafio, na Unity, exigiria criar múltiplas instâncias de `ScriptableObject` via o menu **Create > [Nome do Asset]**, gerado pelo atributo `[CreateAssetMenu]`, preenchendo os mesmos campos e o `enum` de categoria ajustado pelo grupo. A estrutura de dados é equivalente nas duas engines; a diferença está apenas no fluxo de criação de uma nova instância a partir do menu do Editor.

Ao final da semana

Ao final da Semana 6, o projeto do Vertical Slice deve conter:

- O `GameManager` e o `SaveManager` da Semana 4, sem nenhuma alteração.
- O contrato `Interactable`, `Signals`, `Door` e o objeto próprio de cada grupo da Semana 5, sem nenhuma alteração.
- A classe `ItemData` (Resource customizado), com um Enum `Categoria` e campos `nome`, `icone`, `valor`, `descricao` e `categoria` (Encontro 1 e Parte 2 do Encontro 2).
- Um conjunto próprio de três ou mais instâncias `.tres` de itens coletáveis por grupo, salvas em `resources/items/`, coerentes com o tema do próprio Vertical Slice (Parte 3).

Segundo o `PROJECT_ARCHITECTURE.md` (seção 6, Módulo 2), este resultado corresponde à conclusão do item "ItemData (Resource + Enum)" do roadmap, pré-requisito direto de "Pickup" e "Chest" (aplicação do `ItemData` à coleta de itens, via Signal `item_collected`) e de "SaveComponent / SaveData (Resource)", todos construídos na Semana 7.

Desafio

Cada grupo modela seu próprio conjunto de itens coletáveis (baús, moedas, recursos ou equivalente) usando `ItemData` + Enum, com liberdade de categorias e atributos — este é o desafio central da Parte 3. **Entrega: Checkpoint de progresso** do Módulo 2, conforme o Cronograma da Semana 6.

Checklist

- ☐ Enum `Categoria` declarado em `item_data.gd`, com três a cinco valores em PascalCase/maiúsculas
- ☐ Campo `categoria` exportado no `ItemData`, exibido como menu suspenso no Inspector
- ☐ Pelo menos três instâncias `.tres` de itens distintos, com todos os campos preenchidos, incluindo `categoria`
- ☐ Nomes de arquivo `.tres` descrevendo o item concreto, em snake_case
- ☐ Nenhum campo de categoria representado como `String` livre
- ☐ Checkpoint de progresso do Módulo 2 apresentado com sucesso

Glossário

- **Enum:** tipo de dado que restringe um campo a um conjunto fechado e nomeado de valores, eliminando a possibilidade de um valor inválido ou inconsistente.
- **Categoria** : nome do Enum customizado criado nesta semana, aplicado ao campo de categoria do `ItemData` .
- **Menu suspenso (dropdown):** elemento de interface do Inspector gerado automaticamente pelo Godot ao exportar um campo do tipo de um Enum.

Referências

- Godot Documentation — Resources:
<https://docs.godotengine.org/en/stable/tutorials/scripting/resources.html>
- Godot Documentation — GDScript (Enums):
https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/gdscript_basics.html#enums
- Orchestrator — Documentação oficial: <https://orchestrator.cratercrash.space/>
- Unity Manual (consulta comparativa) — ScriptableObjects:
<https://docs.unity3d.com/Manual/class-ScriptableObject.html>
- Canais recomendados para consulta complementar (não substituem a documentação oficial):
GDQuest (<https://www.youtube.com/@GDQuest>), Clear Code
(<https://www.youtube.com/@ClearCode>)